OOPS Output Based Questions

✅ LEVEL 1 — Basic Class & Object (Q1–Q10)

Q1.

```
#include <iostream>
using namespace std;

class A {
public:
   int x = 10;
};

int main() {
   A obj;
   cout << obj.x;
}
```

✅ **Output:** `10`

🎯 *Concept: Basic class property access*

Q2.

```
class A {
public:
   A() { cout << "A "; }
};

int main() {
   A a;
}
```

✅ **Output:** `A`

🎯 *Concept: Constructor call automatically*

Q3.

```
class A {
public:
    A() { cout << 5; }
    ~A() { cout << 10; }
};
int main() {
    A a;
}
```

✅ **Output:** `510`

🎯 *Concept: Destructor auto calls after main scope*

Q4.

```
class A {
public:
    int x;
    A() { x = 7; }
};

int main() {
    A a;
    cout << a.x;
}
```

✅ Output: `7`

Q5.

```
class A {
    int x = 5;
```

```
public:
    void show() { cout << x; }
};

int main() {
    A a;
    a.show();
}
```

✅ Output: `5`

Q6.

```
class A {
public:
    void fun() { cout << "Class A"; }
};

int main() {
    A *a = nullptr;
    a->fun();
}
```

✅ Output: `Class A`

🎯 *Concept: Member functions don't access object memory → safe call on nullptr.*

Q7.

```
class A {
public:
    int x;
    A(int x){ this->x = x; }
};
```

```
int main(){
    A a(20);
    cout << a.x;
}
```

✅ Output: `20`

Q8.

```
class A {
public:
    A(){ cout << "C"; }
};

int main(){
    A a1, a2;
}
```

✅ Output: `CC`

Q9.

```
class A {
public:
    A(){ cout << 1; }
};
int main(){
    A a[3];
}
```

✅ Output: `111`

Q10.

```
class A {
public:
   static int x;
};

int A::x = 50;

int main(){
   cout << A::x;
}
```

✅ Output: `50`

✅ LEVEL 2 — Constructors, Destructors, Copying (Q11–Q20)

Q11.

```
class A {
public:
   A(){ cout << "D"; }
   A(const A&){ cout << "C"; }
};

int main(){
   A a;
   A b = a;
}
```

✅ Output: `DC`

Q12.

```
class A {
public:
    A(){ cout << 1; }
    ~A(){ cout << 2; }
};

int main(){
    A();
}
```

✅ Output: `12`

Q13.

```
class A {
public:
    A(){ cout << "X"; }
};
void fun(){
    static A a;
}
int main(){
    fun();
    fun();
}
```

✅ Output: `X`

🎯 *Static object created once*

Q14.

```
class A {
public:
    A(){ cout << "A"; }
```

```
};
A a;

int main() { cout << "M"; }
```

✅ Output: `A M`

🎯 *Global object constructed before main*

Q15.

```
class A {
public:
    A(){ cout << "1"; }
};

int main(){
    A *a = new A();
    delete a;
}
```

✅ Output: `1`

Q16.

```
class A {
public:
    A(){ cout << "C"; }
    ~A(){ cout << "D"; }
};

int main(){
    A a1;
    {
        A a2;
```

```
    }
}
```

✅ Output: `CCD D`

Actual: `CD D` → final output: **CDD**

Order: a1(C), a2(C), a2(D), a1(D)

Q17.

```
class A {
public:
    A(){ cout << "A"; }
    A(int x){ cout << x; }
};

int main(){
    A a;
    A b(5);
}
```

✅ Output: `A5`

Q18.

```
class A {
public:
    A(){ cout << "0"; }
};
int main(){
    A *a = new A[2];
}
```

✅ Output: `00`

Q19.

```
class A {
public:
    A(){ cout << "K"; }
};

void fun(A obj){}

int main(){
    A a;
    fun(a);
}
```

✅ Output: `KK`

(One for local "a", one for copy inside fun)

Q20.

```
class A {
public:
    A(){ cout << "A"; }
};

A fun(){ A a; return a; }

int main(){
    fun();
}
```

✅ Output: `A`

(RVO optimization avoids extra copy)

✅ LEVEL 3 — Inheritance (Q21–Q30)

Q21.

```
class A {
public:
   A(){ cout << "A"; }
};

class B : public A {
public:
   B(){ cout << "B"; }
};

int main(){
   B b;
}
```

✅ Output: `AB`

Q22.

```
class A {
public:
   A(){ cout << 1; }
};
class B : public A {
public:
   B(){ cout << 2; }
};
class C : public B {
public:
   C(){ cout << 3; }
};

int main(){ C c; }
```

✅ Output: `123`

Q23.

```
class A{
public:
   A(){ cout << "A"; }
};
class B: virtual public A{
public:
   B(){ cout << "B"; }
};
class C: virtual public A{
public:
   C(){ cout << "C"; }
};
class D: public B, public C{
public:
   D(){ cout << "D"; }
};

int main(){ D d; }
```

✅ Output: `ABCD`

Q24.

```
class A{
public:
   A(){ cout << "A"; }
};
class B: public A{
public:
   B(){ cout << "B"; }
};
```

```
int main(){
    A *a = new B();
}
```

✅ Output: `AB`

Q25.

```
class A{
public:
    A(){ cout << "A"; }
    virtual ~A(){ cout << "D"; }
};
class B: public A{
public:
    B(){ cout << "B"; }
    ~B(){ cout << "E"; }
};

int main(){
    A *a = new B();
    delete a;
}
```

✅ Output: `ABE D`

Exact: **ABED**

Q26.

```
class A {
public:
    void fun(){ cout << "A"; }
};
class B: public A {
```

```
public:
    void fun(){ cout << "B"; }
};

int main(){
    B b;
    A *a = &b;
    a->fun();
}
```

✅ Output: `A`

🎯 *Static binding (no virtual keyword)*

Q27.

```
class A{
public:
    virtual void fun(){ cout << "A"; }
};
class B: public A{
public:
    void fun(){ cout << "B"; }
};

int main(){
    A *a = new B();
    a->fun();
}
```

✅ Output: `B`

🎯 *Runtime polymorphism*

Q28.

```
class A {
public:
    A(){ cout << 1; }
};
class B: public A {
public:
    B(){ cout << 2; }
};
int main(){
    B b;
}
```

✅ Output: `12`

Q29.

```
class A{
public:
    A(){ cout << "X"; }
};
class B: public A {
public:
    B(){ cout << "Y"; }
};

int main(){
    A *a = new B();
}
```

✅ Output: `XY`

Q30.

```
class A{
public:
```

```
    virtual void show(){ cout << "A"; }
};
class B: public A{
public:
    void show(){ cout << "B"; }
};
void display(A a){ a.show(); }

int main(){
    B b;
    display(b);
}
```

✅ Output: `A`

🎯 *Object slicing*

✅ LEVEL 4 — Polymorphism, Overloading, Overriding (Q31–Q40)

Q31.

```
class A{
public:
    void fun(int){ cout << 1; }
    void fun(double){ cout << 2; }
};

int main(){
    A a;
    a.fun(5.0);
}
```

✅ Output: `2`

Q32.

```
class A{
public:
   virtual void fun(int x){ cout << 1; }
};
class B: public A{
public:
   void fun(double x){ cout << 2; }
};

int main(){
   B b;
   b.fun(5);
}
```

✅ Output: `2`

🎯 *Hidden base function (not overridden)*

Q33.

```
class A{
public:
   virtual void fun(){ cout << "A"; }
};
class B: public A{
};
int main(){
   B b;
   b.fun();
}
```

✅ Output: `A`

Q34.

```
class A{
public:
   virtual void fun(){ cout << "A"; }
};
class B: public A{
public:
   void fun(){ cout << "B"; }
};
int main(){
   A a;
   B b;
   a = b;
   a.fun();
}
```

✅ Output: `A`

🎯 *Slicing kills polymorphism*

Q35.

```
class Base {
public:
   Base(){ show(); }
   virtual void show(){ cout << "B"; }
};

class Derived: public Base {
public:
   Derived(){ show(); }
   void show(){ cout << "D"; }
};

int main(){ Derived d; }
```

✅ Output: `BD`

🎯 *Virtual calls inside constructor → base version only*

Q36.

```
class A{
public:
   int x=5;
};
class B: public A{
public:
   int x=10;
};

int main(){
   B b;
   cout << b.x;
}
```

✅ Output: `10`

🎯 *Variable hiding*

Q37.

```
class A{
public:
   void show(){ cout << 1; }
};
class B: public A{
public:
   void show(int x){ cout << 2; }
};

int main(){
   B b;
```

```
    b.show();
}
```

✅ Output: Error?

No — base function hidden?

Yes — but can be accessed via cast.

✅ Output: `1`

Q38.

```
class A{
public:
    virtual void show(){ cout << 1; }
};
class B: public A{
public:
    void show(){ cout << 2; }
};
int main(){
    A *a = new B;
    B *b = dynamic_cast<B*>(a);
    b->show();
}
```

✅ Output: `2`

Q39.

```
class A{
public:
    A(){ cout << "A"; }
};
class B{
public:
```

```
    B(){ cout << "B"; }
};
class C: public A, public B{ };

int main(){ C c; }
```

✅ Output: `AB`

Q40.

```
class A{
public:
    A(){ cout << 1; }
};
class B: virtual public A{
public:
    B(){ cout << 2; }
};
class C: virtual public A{
public:
    C(){ cout << 3; }
};
class D: public B, public C{
public:
    D(){ cout << 4; }
};

int main(){ D d; }
```

✅ Output: `1234`

✅ LEVEL 5 — Advanced + Tricky (Q41–Q50)

Q41.

```
class A{
public:
    A(){ cout << "A"; }
    ~A(){ cout << "D"; }
};

void fun(){
    static A a;
}

int main(){
    fun();
    fun();
}
```

✅ Output: `A D`

(D printed at program end)

Q42.

```
class A{
public:
    A(){ cout << 1; }
    ~A(){ cout << 2; }
};

int main(){
    A *a = new A;
    return 0;
}
```

✅ Output: `1`

(Leak → destructor not called)

Q43.

```
class A{
public:
    A(){ cout << "X"; }
};

A fun() {
    static A a;
    return a;
}

int main(){
    fun();
}
```

✅ Output: `X`

Q44.

```
class A{
public:
    A(){ cout << "C"; }
};
A a;

int main(){}
```

✅ Output: `C`

Q45.

```
class A{
public:
    A(){ cout << "C"; }
```

```
} a1;

class B{
public:
   B(){ cout << "D"; }
} a2;

int main(){}
```

✅ Output: `CD`

Q46.

```
class A{
public:
   A(){ cout << 1; }
};
class B: public A{
public:
   B(){ cout << 2; }
};

void fun(A a){}

int main(){
   B b;
   fun(b);
}
```

✅ Output: `12 1`

Explanation:

b → prints `12`

copy into A → prints `1`

Q47.

```
class A{
public:
   virtual void show(){ cout << "A"; }
};
class B: public A{
public:
   void show(){ cout << "B"; }
};
int main(){
   B b;
   A &a = b;
   a.show();
}
```

✅ Output: `B`

Q48.

```
class A{
public:
   A(){ cout << "A"; }
};
class B: public A{
public:
   B(){ cout << "B"; }
};
class C: public B{
public:
   C(){ cout << "C"; }
};

int main(){
   A *a = new C();
}
```

✅ Output: `ABC`

Q49.

```
class A{
public:
   A(){ cout << "A"; }
};
class B: public A{
public:
   static B obj;
   B(){ cout << "B"; }
};

B B::obj;

int main(){}
```

✅ Output: `AB`

Q50.

```
class A{
public:
   A(){ cout << 1; }
};
class B: public A{
public:
   B(){ cout << 2; }
};
int main(){
   A *a = new B;
   delete a;
}
```

✅ Output: `12`

(*Destructor missing → UB but output stays* `12`)